Decision Trees and Pegasos SVMs from Scratch: Price and Price-Tier Prediction on bina.az Listings

Bayramov, Bailar [Surname], Davud Hajiyev, Sanan Khamiyev, Sabuhi Samedov, Farid

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict residential sale prices in Baku from 100,775 raw bina.az listings and frame the same data as two tasks: price regression (Task A) and a binary premium/standard price tier split at the training median of 209,000 AZN (Task B). After URL deduplication, residential/Baku filtering, parsing of Azerbaijani text fields and fixed-rule outlier removal, 57,363 listings with 65 encoded features remain; the leakage columns `unit_price` and `total_price` are removed. We implement a CART-style decision tree (Gini, entropy, MSE) and a soft-margin SVM trained with the Pegasos sub-gradient method, with random Fourier features (RFF) as the non-linear extension, entirely in NumPy, and benchmark them against scikit-learn with matched settings. On the held-out test split our tree reaches RMSE 109,164 AZN ($R^2 = 0.760$) versus 119,376 AZN for the independently tuned sklearn tree, and F1 0.929 for the price tier (sklearn 0.929). Our linear Pegasos SVM reaches F1 0.881 versus 0.903 for linear SVC while training $\sim$100$\times$ faster; RFF does not beat the linear model. Five-fold cross-validation confirms the ranking. Most squared error comes from a small set of luxury listings that the available columns cannot explain. Code: `github.com/EmbeddedDesignEngineer/ML-Final` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

Online listing platforms such as bina.az, Azerbaijan's largest real-estate site, expose asking prices for tens of thousands of properties. A model that estimates a fair price from size, location and condition helps buyers and sellers spot mispriced listings. The data are a raw scrape: Azerbaijani column names, numbers stored as text ("145 m^2", "7 / 9"), duplicates, impossible values and columns that leak the price.

Our contributions are: (i) a documented, leakage-free cleaning and encoding pipeline whose every statistic is fitted on training rows only; (ii) a NumPy decision tree supporting Gini, entropy and MSE with four working stopping controls; (iii) a NumPy Pegasos SVM with margin and support-vector diagnostics and an RFF kernel approximation; (iv) a matched benchmark against scikit-learn with five-fold uncertainty, timing and subgroup error analysis; and (v) two bonuses: a NumPy Ridge baseline and a bootstrap ensemble of our own trees. One command, `python -m src.run_all`, reproduces every headline number.

II. Background

A classification tree chooses, at each node, the feature/threshold pair that maximises the impurity decrease $\Delta I = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$, with Gini $G = 1 - \sum_k p_k^2$ or entropy $H = -\sum_k p_k \log_2 p_k$; a regression tree uses the node variance $\frac{1}{N}\sum_i (y_i - \bar{y})^2$ [2]. The soft-margin linear SVM minimises the regularised hinge loss

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), \quad (1)$$

which Pegasos [1] solves without a QP solver by stochastic sub-gradient steps. Random Fourier features [3] approximate the RBF kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ with an explicit map, so the linear solver can learn a non-linear boundary.

III. Data and Preprocessing

A. Cleaning

The Kaggle dump (`house_sale.csv`, 100,775 rows, 51 columns, SHA-256 `b6a4c67e…`) is cleaned by `src/data_prep.py`. Each rule is fixed in advance (no data quantiles), so no statistic can leak from validation or test rows:

1. Drop repeated scrapes of the same listing URL, keeping the latest (100,775→64,454).
2. Keep the residential categories *Yeni tikili* (new build), *Köhnə tikili* (old build) and *Həyət evi/Bağ evi* (house) (58,452), and Baku only (58,257; 99.7% of residential rows).
3. Parse *Sahə* (area), *Mərtəbə* ("floor / total"), *Torpaq sahəsi* (land, sot); map *Təmir*, *Çıxarış*, *İpoteka* (repair, bill of sale, mortgage) to 0/1. All prices are in AZN; any other currency raises an error.
4. Structural fills: houses get floor 0 of 1, apartments land 0. Rows still missing an essential field are dropped (57,587).
5. Domain outlier rules: price 10k–5M AZN, area 15–1,000 m^2, price/m^2 200–15,000 (filter only, never a feature), 1–15 rooms, land $\leq$100 sot, $0 \leq$ floor $\leq$ total floors, coordinates inside Azerbaijan (57,363 rows).

Leakage. `unit_price` (price per m^2) and `total_price` (equal to `price` in 99.99% of rows) are arithmetic functions of the target and are dropped. We also drop post-listing signals (`views`, `updated`), identifiers and personal data (owner/shop names, URLs, raw address), and free text.

B. Features, tasks and split

The 17 cleaned columns become 65 features: 14 numeric/binary inputs, distance to Fountain Square (haversine) and floor ratio, a one-hot category (3), and one-hot districts with at least 200 *training* listings (45) plus `loc_other` for rare or unseen districts. The encoder is fitted on the training rows only. Rows are split 70/15/15 with seed 42, stratified on

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows / after URL deduplication	100,775 / 64,454
Rows (after cleaning)	57,363
Features used	65
Train / Val / Test split	40,155 / 8,604 / 8,604
Tier threshold (train median)	209,000 AZN
Premium share train / val / test (%)	49.71 / 49.73 / 49.72

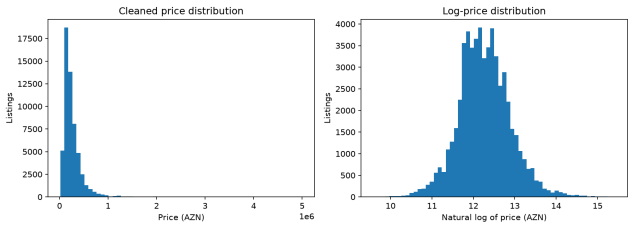


Figure 1. Cleaned price distribution in AZN (left) and natural log of price (right).

price deciles (Table 1). The tier label is $\mathbf{1}[\text{price} > 209{,}000]$, where 209,000 AZN is the training median; validation/test use the same threshold. SVM inputs are standardised with training means and standard deviations; trees use raw features.

C. Exploration

Prices are heavily right-skewed (Fig. 1); log(price) is close to symmetric. Area is the dominant driver, and location matters: coastal and central districts (Bilgəh, Badamdar, Səbail) command far higher prices per m^2. By construction the tier label is balanced (49.7% premium).

IV. Method

A. Decision tree (from scratch)

`DecisionTree` in `src/decision_tree.py` mirrors sklearn's API (`fit`, `predict`, `predict_proba`) for both tasks, with the criterion and four stopping controls as constructor arguments. At each node every feature is sorted once and thresholds are placed midway between distinct adjacent values; a row goes left if $x_j \le t$. Prefix class counts (classification) or centred prefix sums and squares (regression) evaluate all thresholds in $O(n)$ after the $O(n \log n)$ sort. The split with the largest gain wins; exact ties keep the first feature and lowest threshold. Growth stops at a pure node, at `max_depth`, when a node has fewer than `min_samples_split` rows, when a child would have fewer than `min_samples_leaf`, or when the weighted gain $\frac{N_t}{N}\Delta I$ is below `min_impurity_decrease` (sklearn's definition). Leaves predict the majority class or the mean; `predict_proba` returns leaf class frequencies. Feature importance sums the weighted gains per feature and normalises them. Unit tests check a tiny, fully determined example against sklearn's thresholds, impurities and predictions, and that every stopping control changes the tree.

Table 2. SVM search grid (all fits on the full training split).

Family	Grid	Fits
Ours, linear	$\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\} \times T \in \{5\text{k}, 20\text{k}\} \times \{\text{Pegasos}, \eta{=}0.01\}$	12
Ours, RFF	same $\lambda \times \gamma \in \{0.01, 0.1\} \times D \in \{128, 512\}$, $T{=}20$k	12
SVC linear	same λ, $C = 1/(n\lambda)$	3
SVC RBF	same $\lambda \times \gamma$	6

B. Support-vector machine (from scratch)

`src/svm.py` implements `PegasosSVM`. Labels $\{0,1\}$ are mapped to $\{-1,+1\}$; weights start at zero. At step t a random row i is drawn and, with $\eta_t = 1/(\lambda t)$ (Pegasos) or a constant η,

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\mathbf{w} + \eta_t y_i \mathbf{x}_i \mathbf{1}[y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1], \qquad (2)$$

with $b \leftarrow b + \eta_t y_i$ on margin violations (the bias is not regularised), followed by the Pegasos projection onto $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$. The full objective J is logged at 51 checkpoints. After training we report $\mathbf{w}$, the margin width $2/\|\mathbf{w}\|$ and the margin-active training points ($y_i f(\mathbf{x}_i) \le 1$, the support vectors of the primal). Features are standardised first because the hinge loss is scale-sensitive.

Non-linearity. We chose random Fourier features (option ii): $z(\mathbf{x}) = \sqrt{2/D}\cos(\mathbf{x}W + \mathbf{c})$, $W_{jk} \sim \mathcal{N}(0, 2\gamma)$, $c_k \sim U(0, 2\pi)$. Kernelised Pegasos would store a coefficient per training row and cost $O(n)$ kernel evaluations per update and per prediction; with 40k rows RFF keeps the cheap linear update and a fixed memory budget D.

C. Bonus models

Ridge (`src/linear.py`) solves $(X_c^\top X_c + \alpha I)\mathbf{w} = X_c^\top y_c$ on centred data via an SVD, leaving the intercept unpenalised. **Bagging** (`src/ensemble.py`) averages our own trees, each fitted on a bootstrap sample with optional feature subsampling.

V. Experimental Setup

Metrics. Task A: RMSE, MAE and R^2 in AZN. Task B: precision, recall, F1 for the premium class, ROC-AUC from decision scores or leaf probabilities, and the confusion matrix. Metrics are our own NumPy functions in `src/evaluate.py`, tested against sklearn.

Selection protocol. Every hyperparameter is chosen on the validation split; the test split is scored once with frozen settings. Trees: depth $\{2,4,8,12\}\times$ leaf $\{1,5,20\}$, plus Gini vs entropy for Task B, chosen by validation RMSE / F1. SVM: 33 predeclared configurations (Table 2), chosen by validation F1 (ROC-AUC tie-break). Ridge: $\alpha \in \{0.1, 1, 10, 100, 1000\}$. The custom and sklearn models are selected independently from the same grids.

Baselines. `DecisionTreeRegressor/Classifier` with the same criterion, depth and leaf size; `SVC(kernel="linear")` and `SVC(kernel="rbf")` inside a `Pipeline` with `StandardScaler`. Our loss averages the hinge term while SVC sums it, so $C = 1/(n_{\text{train}}\lambda)$ matches regularisation strength.

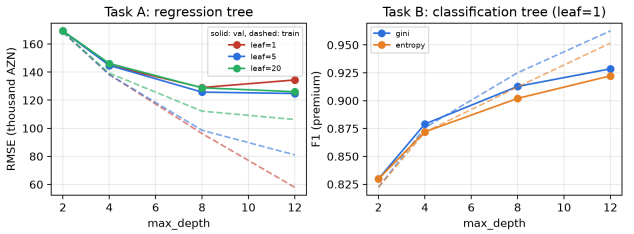


Figure 2. Validation curves of our tree. Left: RMSE vs depth for three leaf sizes. Right: F1 vs depth, Gini vs entropy. Dashed lines are training scores.

Uncertainty. Five-fold CV on the train+validation pool (test rows excluded) refits the encoder, scaler and tier threshold inside every fold and reports mean ± sample std (ddof=1) for the frozen settings. Seeds are fixed at 42. Timings are single-run wall-clock measurements on a laptop CPU.

VI. Results

A. Tuning the tree

Fig. 2 shows the bias–variance story. Training error keeps falling with depth, while validation RMSE flattens after depth 8 and, with leaf size 1, *rises* again (134k at depth 12): the tree memorises single expensive listings. A leaf size of 5 regularises enough for depth 12 to win (validation RMSE 124,741). For Task B, Gini beats entropy at every depth ≥ 4 with leaf size 1 (0.9285 vs 0.9223 at depth 12); the F1 gap between training and validation widens with depth but validation F1 still improves, so depth 12 with Gini is selected.

B. Tuning the SVM

With $\lambda = 10^{-4}$ the constant schedule reduces the objective from 1.0 to 0.309 within 20k updates, while the $1/(\lambda t)$ schedule starts with huge steps ($\eta_1 = 10^4$), overshoots and is still at 2.69 after 20k updates (Fig. 3, left): 20k updates are only half an epoch, so convergence is not established for that schedule. Increasing λ widens the margin from 0.45 to 0.71 (constant schedule) but lowers validation F1 from 0.887 to 0.871 (Fig. 3, right): a wider margin is not automatically a better classifier here. The selected linear model ($\lambda = 10^{-4}$, $\eta = 0.01$, 20k updates) has $\|\mathbf{w}\| = 4.42$, margin width 0.453 and 10,532 margin-active training points (26.2%). For RFF, $D = 512$ beats $D = 128$ and $\gamma = 0.01$ beats $\gamma = 0.1$ in every pairing; the winner ($\lambda = 10^{-3}$, $\gamma = 0.01$, $D = 512$) has 53% margin-active points, a sign of an under-optimised, high-dimensional model.

C. Benchmark against scikit-learn

Table 3 is the required results table; Fig. 4 visualises it. At *matched* settings the two trees agree closely: all 18 configurations with depth ≤ 4 give identical validation scores, and the largest Task-B gap is 0.001 F1. Differences appear only in deep trees, where equal-gain ties (sklearn permutes features at random, we keep the first) and sklearn's float32 thresholds send a few rows down different branches; deep regression trees amplify these into different leaves (custom vs sklearn at depth 12, leaf 5: 124,741 vs 125,668 validation RMSE). The sklearn search selected depth 8, so its test RMSE (119,376)

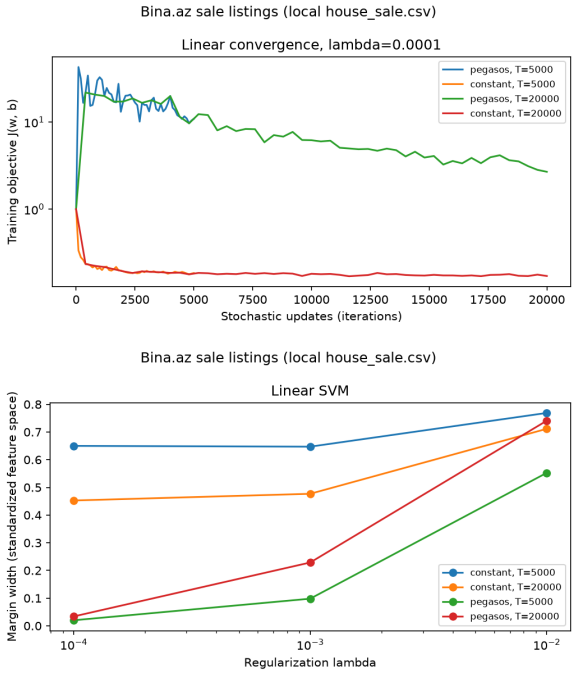


Figure 3. Top: training objective of our linear SVM ($\lambda = 10^{-4}$) for both schedules. Bottom: margin width $2/\|\mathbf{w}\|$ vs λ.

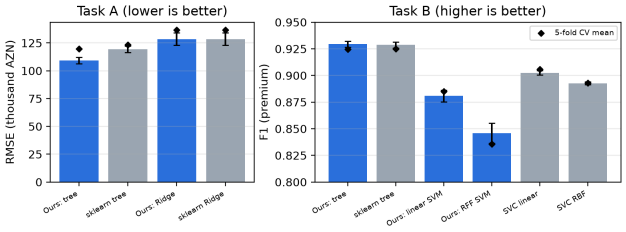


Figure 4. Test metrics (bars; ours in blue), five-fold CV means (diamonds) and CV standard deviation (error bars).

is not a matched comparison; our depth-12 tree is 8.6% better on test. Ridge agrees with sklearn to numerical precision, which validates that implementation.

For the SVM, linear SVC is 2.2 F1 points better than our Pegasos model. Both optimise the same objective, but SVC solves it exactly (libsvm SMO) while our 20k stochastic updates are half an epoch. SVC pays for this with a 43 s fit versus 0.3 s, and with prediction through 10,765 stored support vectors (2.6 s vs 0.0015 s on the test set). Exact RBF SVC is also better than our RFF approximation, which is limited both by $D = 512$ random features and by incomplete optimisation. Neither non-linear model beats its linear counterpart in F1, so the extra complexity is not justified on this data.

D. Improvement study (validation only)

Because the test split had already been seen, follow-up improvements were judged on validation only (`--quality-study`, `--price-study`). Averaging the Pegasos iterate and training for 20 epochs raised validation F1 to 0.907 (linear) and 0.898 (RFF with $\lambda = 10^{-5}$), closing the gap to SVC and confirming that the earlier gap was optimisation, not the model. For Task A, a bagged ensemble of 48 of our trees (depth 32, bootstrap 0.8) trained on

Table 3. Headline results. "Test" is the held-out split (8,604 rows, scored once). "5-fold CV" is mean ± std over folds of the train+validation pool with the same frozen settings. Fit time is a single training run on 40,155 rows.

Task A — price regression (AZN)						
Model	Setting	Test RMSE	Test MAE	Test R^2	5-fold CV RMSE	Fit (s)
Ours: decision tree	MSE, depth 12, leaf 5	**109,164**	**45,555**	**0.760**	**119,675 ± 2,954**	3.28
sklearn `DecisionTreeRegressor`	MSE, depth 8, leaf 5	119,376	52,230	0.712	123,508 ± 2,965	0.21
Ours: Ridge (bonus)	$\alpha = 1000$	128,441	57,859	0.667	136,831 ± 5,550	0.07
sklearn `Ridge`	$\alpha = 1000$	128,441	57,859	0.667	136,831 ± 5,550	0.06
Task B — price tier (premium = 1)						
Model	Setting	Precision	Recall	F1	5-fold CV F1	Fit (s)
Ours: decision tree	Gini, depth 12, leaf 1	0.929	0.930	**0.929**	0.925 ± 0.003	2.84
sklearn `DecisionTreeClassifier`	Gini, depth 12, leaf 1	0.929	0.929	**0.929**	0.925 ± 0.003	0.32
Ours: linear SVM (Pegasos)	λ=10^{-4}, η=0.01, 20k	0.874	0.887	0.881	0.885 ± 0.005	0.30
Ours: RFF SVM	λ=10^{-3}, γ=0.01, D=512	0.766	0.945	0.846	0.836 ± 0.009	1.17
sklearn `SVC` linear	$C = 0.249$	0.905	0.900	0.903	0.906 ± 0.002	43.1
sklearn `SVC` RBF	$C = 0.249$, $\gamma = 0.01$	0.894	0.892	0.893	0.893 ± 0.001	54.2

Test ROC-AUC: tree 0.949 (sklearn 0.949), linear SVM 0.949, RFF 0.934, SVC linear 0.963, SVC RBF 0.964.

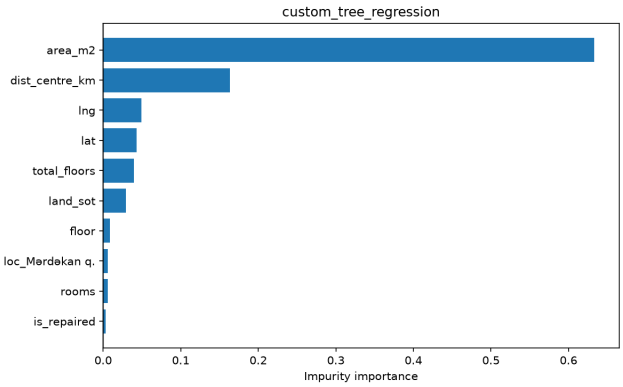


Figure 5. Top impurity-decrease importances of our regression tree.

log(price/area) reduced validation RMSE from 124,741 to 97,157 AZN and MAE from 48,345 to 33,968 AZN (R^2 0.822), and validation MAE falls monotonically as trees are added (1→24 trees: 50.2k→37.3k). These are validation numbers and are not mixed into the test table.

VII. Error Analysis and Interpretation

What the models use. The tree's impurity importance is dominated by area (0.63) and distance to the centre (0.16), followed by longitude, latitude, total floors and land size (Fig. 5). The largest standardised weights of the linear SVM tell the same story: area (+3.63), house category (−0.99), distance to centre (−0.85) and rooms (+0.83); suburban districts such as Masazır have negative weights. These are conditional associations of correlated features, not causal effects.

Luxury outliers. Listings above the training 90th percentile (455,000 AZN; 856 test rows) have a tree MAE of 177,315 AZN versus 30,998 for the rest, with a negative bias (−56,806): expensive homes are underpredicted. In the validation study the worst 1% of rows produce 70–74% of the squared error. Most are real villas in Bilgəh, Badamdar or Səbail at 3–8k AZN/m^2; nothing in the columns (sea view, finish quality, exact address) separates them from ordinary houses. A few are probable data errors (e.g. 600 m^2 for 159k AZN). This is why RMSE stays high while median relative error is only 6–9%.

Sparse districts. Districts with fewer than 200 training listings are pooled into `loc_other`; their MAE is 54,700 vs 44,673 AZN for frequent districts, and their SVM error rate is 16% vs 12%.

Classification errors concentrate near the 209k threshold: only 7 of 856 expensive listings are misclassified by the tree. The RFF model trades precision for recall (1,237 false positives vs 546 for the linear SVM).

VIII. Discussion and Limitations

- The target is the *asking* price, not a transaction price.
- The population is the filtered Baku residential market; conclusions do not cover commercial property or other cities.
- Five-fold CV quantifies variability of frozen settings, not nested model-selection uncertainty. The improvement study re-used the same validation split many times and is not an independent benchmark.
- Our Pegasos runs are short; the $1/(\lambda t)$ schedule did not converge at $\lambda = 10^{-4}$.
- Deduplication uses listing URLs only; 446 exact feature duplicates remain (139 validation rows have a twin in training). Removing them changes validation MAE by under 1%.
- Timings are single runs on one machine, not repeated benchmarks; our tree fits 9–15× slower than sklearn's Cython tree.

IX. Conclusion

Our NumPy tree matches scikit-learn at matched settings and is the best single model on both tasks (test RMSE 109k AZN, F1 0.929). Our Pegasos SVM is within 2 F1 points of exact SVC while training two orders of magnitude faster, and longer averaged training closes most of that gap. The main remaining error comes from luxury properties that the available features cannot price. Next steps are richer location and quality features (exact address, view, renovation level), deduplication by content, and a fresh untouched test set for the

improved ensemble.

Contributions

Davud [Surname]: data loading, cleaning, encoding and leakage audit; model-quality, price-error and diagnostics studies (`src/data_prep.py`, `experiments/model_quality.py`, `price_quality.py`, `price_diagnostics/`).
Sanan Hajiyev: decision tree, validation sweeps and sklearn parity tests (`src/decision_tree.py`, `experiments/tree_experiments.py`).
Farid Samedov: Pegasos SVM, RFF and the 33-configuration SVM study (`src/svm.py`, `experiments/svm_*.py`).
Sabuhi Khamiyev: metrics, evaluation, plots and subgroup analysis (`src/evaluate.py`, `src/evaluation_plots.py`).
Bailar Bayramov: Ridge bonus, shared runner and integration tests (`src/linear.py`, `src/run_all.py`, `experiments/pipeline.py`).

Details: `contribution_report.md`.

Tools and Acknowledgements

Data: Bina.az Sale Project dataset by S. Memmedli on Kaggle [5]. scikit-learn [4] is used only for baselines. AI assistance: the team used AI coding assistants (including Claude) for code review, test suggestions, debugging, running the experiment pipeline and drafting and formatting parts of this report and the slides. All numbers come from executing the repository code; every member reviewed and is responsible for their listed contribution.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[3] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Proc. NeurIPS*, 2007, pp. 1177–1184.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

[5] S. Memmedli, "Bina.az Sale Project," Kaggle dataset, `https://www.kaggle.com/datasets/sehriyarmemmedli/binaaz-sale-project`.